

SAT- und MaxSAT-Enkodierung des Art Gallery Problems

Methodik, Implementierung und Benchmark-Ergebnisse

Elena Berschauer

17. Juni 2026

Lehrstuhl für Rechnerarchitektur

Betreuer: Mathias Fleury

Art Gallery Problem

Pipeline-Überblick

Diskretisierung & Kandidaten

Sichtbarkeit

SAT-Encoding

Iteratives Lösen mit Kissat

MaxSAT-Encoding

Polygon-Klassen

Ergebnisse

Diskussion

Fazit & Ausblick

Art Gallery Problem

Warum AGP lösen?

Art Gallery Problem (AGP): Wie viele Wächter (Guards) braucht man, um ein Polygon vollständig zu überwachen?

Praktische Relevanz

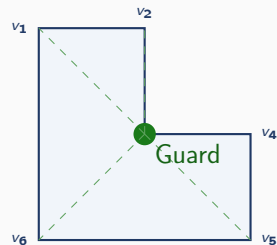
Kameraplatzierung, Sensornetze,
Roboternavigation

Theoretische Tiefe

Klassisches Problem der Computational
Geometry, **NP-schwer** (Lee & Lin, 1986)

Ziel

Das *Minimum* an Guards finden, beweisbar
optimal, nicht nur eine obere Schranke. $\Rightarrow$ SAT



L-Polygon: 1 Guard genügt

Pipeline-Überblick



- **Candidates:** Eckpunkte + relevante Kantenschnittpunkte
- **Visibility:** Sichtbarkeitsmatrix V über Sub-Segment-Dekomposition
- **Encoding:** Fokus dieses Vortrags
- **Solving:** Kissat (iterativ) vs. MaxHS (direkt)

Diskretisierung & Kandidaten

Diskretisierung: Warum nicht kontinuierlich?

Problem

Das AGP ist über kontinuierliche Flächen definiert — SAT kennt nur endlich viele boolesche Variablen.

Lösung: Diskretisierung

- Wähle endliche Kandidatenmenge $P \subset \mathbb{R}^2$
- Guards und Witnesses stammen beide aus P
- Coverage wird relativ zu P definiert: jeder Punkt in P muss von mindestens einem Guard gesehen werden

Guards = Witnesses

In dieser Implementierung ist P sowohl die Menge möglicher Guard-Positionen als auch die Menge der zu überwachenden Witness-Punkte.

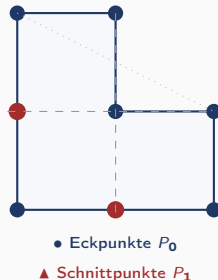
Konsequenz: Sichtbarkeitsmatrix V ist *symmetrisch*, $V[i][i] = 1$.

Welche Punkte werden gewählt?

1. Alle Eckpunkte des Polygons (P_0)
2. Nur *sichtbare* Eckenpaare v_i, v_j
betrachten: $\overline{v_i v_j} \subseteq \bar{P}$
3. Schnittpunkte dieser Sichtlinien mit
Kanteninneren werden zu P_1
hinzugefügt

Warum nur sichtbare Paare?

Eine Gerade durch zwei Ecken, die sich nicht sehen, verlässt das Polygon — ihre Schnittpunkte sind geometrisch nicht relevant für die Sichtbarkeitsstruktur.

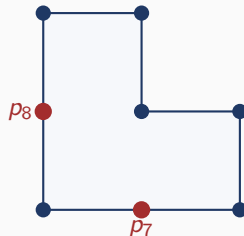


L-Polygon: 15 Eckenpaare $\rightarrow$ 8 Kandidaten

1. $\binom{6}{2} = 15$ mögliche Eckenpaare
2. Mittelpunktstest: nur **12** Paare sichtbar
(3 verlassen das Polygon, z. B. $v_1 \rightarrow v_4$)
3. Von den 12 Sichtlinien liefern nur **2** neue Schnittpunkte im Kanteninneren

L-Polygon: finale Kandidatenmenge

Quelle	Anzahl
Eckpunkte P_0	6
Schnittpunkte P_1	2
$ P $ gesamt	8



$p_1 - p_6$: Eckpunkte

p_7, p_8 : Schnittpunkte

$p_7 = (1, 0)$: Linie $v_2 v_3 \cap e_5$

$p_8 = (0, 1)$: Linie $v_3 v_4 \cap e_6$

Sichtbarkeit

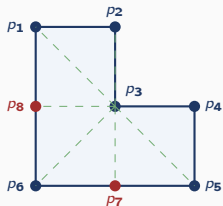
Definition

Guard p_i sieht Witness p_j genau dann, wenn die Strecke $\overline{p_i p_j}$ vollständig in der abgeschlossenen Hülle $\bar{P} = P^\circ \cup \partial P$ liegt.

Wichtige Randfälle:

- **Kanten sind sichtbar:** Strecken entlang des Randes zählen als sichtbar ($\bar{P}$ ist abgeschlossen)
- **Reflex-Vertex:** Sichtlinie darf einen Reflex-Vertex berühren, solange sie das Polygoninnere nicht verlässt — in dieser Implementierung gilt das als sichtbar
- **Numerische Toleranz:** $\varepsilon = 10^{-6}$ für Floating-Point-Vergleiche

Sichtbarkeitsmatrix V — L-Polygon ($|P| = 8$)



Guard p_3 sieht alle

$|P| = 8$
 (p_1 – p_6 Ecken, p_7, p_8
 Schnittpunkte)

$$V[i][j] = 1 \iff \overline{p_i p_j} \subseteq \bar{P}$$

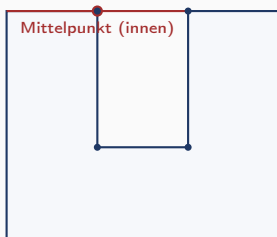
	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8
p_1	1	1	1	0	1	1	1	1
p_2	1	1	1	0	0	1	1	1
p_3	1	1	1	1	1	1	1	1
p_4	0	0	1	1	1	1	1	1
p_5	1	0	1	1	1	1	1	1
p_6	1	1	1	1	1	1	1	1
p_7	1	1	1	1	1	1	1	1
p_8	1	1	1	1	1	1	1	1

p_3, p_6, p_7, p_8 : vollständige Zeile $\Rightarrow$ sehen alle 8 Punkte

$V[1][4] = 0$: $p_1 \rightarrow p_4$ verlässt das Polygon

Naiver Mittelpunktstest: Falsch positiv

Segment von $(0, 5)$ zu $(4, 5)$ im
U-Polygon:

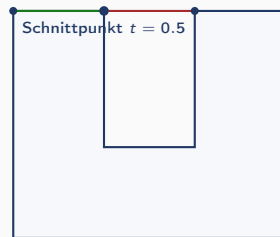


Mittelpunkt $(2, 5)$ liegt innen $\Rightarrow$ sichtbar

Aber: Segment geht durch Lücke!

Sub-Segment-Dekomposition

Zerlegung an Schnittpunkten mit
Polygonrand:



Mid2 bei $t = 0.75$: $(3, 5)$ liegt außen
 $\Rightarrow$ **nicht sichtbar** — korrekt

SAT-Encoding

Guard-Variable

Für jeden Kandidaten $p_i \in P$:

$$g_i \in \{0, 1\}, \quad g_i = 1 \iff p_i \text{ wird als Guard eingesetzt}$$

Coverage-Klausel (pro Witness p_j)

$$\forall j : \bigvee_{V[i][j]=1} g_i$$

Mindestens ein Guard, der p_j sieht, muss aktiv sein.

Coverage allein reicht nicht

Triviale Lösung: alle $g_i = 1$ erfüllt alle Klauseln. $\Rightarrow$ Kardinalitätsbeschränkung nötig.

Kardinalitätsbeschränkung: $\sum g_i \leq k$

- SAT kennt keine Summation — nur boolesche Disjunktionen
- Die Schranke muss also über Klauseln simuliert werden

Sequential Counter Encoding

Simuliert einen Zähler über Hilfsvariablen $s_{i,j}$.

$\mathcal{O}(n \cdot k)$ zusätzliche Variablen und Klauseln — skaliert linear in n und k , allgemein einsetzbar.

⇒ Diese Arbeit verwendet den **Sequential Counter**.

Hilfsvariable

$s_{i,j} = 1 \iff$ unter den ersten i Kandidaten sind mindestens j Guards aktiv

Beispiel: $s_{5,2} = 1$ bedeutet: unter $g_1, \dots, g_5$ sind mindestens 2 Guards aktiv.

Initialisierung ($i = 1$): Äquivalenz $s_{1,1} \leftrightarrow g_1$

$$g_1 = 1 \Rightarrow s_{1,1} = 1$$

Implikation: $g_1 \rightarrow s_{1,1}$

$$s_{1,1} = 1 \Rightarrow g_1 = 1$$

Implikation: $s_{1,1} \rightarrow g_1$

$$\neg g_1 \vee s_{1,1}$$

$$\neg s_{1,1} \vee g_1$$

Beide Klauseln zusammen erzwingen die Äquivalenz — ohne Richtung 2 könnte der Solver $s_{1,1}$ frei setzen.

Herleitung: Monotonie & Inkrement

Monotonie

Regel: Sind unter den ersten $i-1$ Kandidaten schon $\geq j$ Guards aktiv, gilt das auch für die ersten i .

Implikation: $s_{i-1,j} \rightarrow s_{i,j} \quad (a \rightarrow b \equiv \neg a \vee b)$

$$\neg s_{i-1,j} \vee s_{i,j}$$

Inkrement

Regel: Sind unter den ersten $i-1$ Kandidaten $\geq j-1$ Guards aktiv *und* wird g_i zusätzlich gewählt, sind es $\geq j$.

Implikation: $s_{i-1,j-1} \wedge g_i \rightarrow s_{i,j} \quad ((a \wedge b) \rightarrow c \equiv \neg a \vee \neg b \vee c)$

$$\neg s_{i-1,j-1} \vee \neg g_i \vee s_{i,j}$$

Herleitung: Obergrenze

Obergrenze

Regel: Mehr als k Guards sind verboten. Sind unter den ersten $i-1$ Kandidaten schon k Guards aktiv ($s_{i-1,k} = 1$), darf g_i nicht mehr aktiviert werden.

Implikation: $s_{i-1,k} \rightarrow \neg g_i \quad \equiv \quad \neg(s_{i-1,k} \wedge g_i)$

$$\neg s_{i-1,k} \vee \neg g_i$$

Das ist die eigentliche Schranke $\sum_i g_i \leq k$.

Idee	Implikation	CNF
Monotonie	$s_{i-1,j} \rightarrow s_{i,j}$	$\neg s_{i-1,j} \vee s_{i,j}$
Inkrement	$s_{i-1,j-1} \wedge g_i \rightarrow s_{i,j}$	$\neg s_{i-1,j-1} \vee \neg g_i \vee s_{i,j}$
Obergrenze	$s_{i-1,k} \rightarrow \neg g_i$	$\neg s_{i-1,k} \vee \neg g_i$

Initialisierung wurde bereits gezeigt — Äquivalenz $s_{1,1} \leftrightarrow g_1$ macht den Start des Zählers

Iteratives Lösen mit Kissat

1. Starte mit $k = 1$
2. Erzeuge CNF: Coverage-Klauseln $\wedge$ Sequential Counter ($\leq k$)
3. Kissat löst die Instanz
4. Falls **UNSAT**: $k \leftarrow k + 1$, zurück zu Schritt 2
5. Erstes **SAT**-Ergebnis $\Rightarrow k$ ist minimal

MaxSAT-Encoding

Motivation

Iteratives SAT: im schlimmsten Fall $k_{\min}$ Solver-Aufrufe, jeder UNSAT-Nachweis teurer als der vorherige.

MaxSAT formuliert Minimierung direkt:

- **Hard Clauses:** müssen erfüllt sein — Coverage-Klauseln
- **Soft Clauses:** sollen erfüllt sein, mit Gewicht — Verletzungen werden minimiert

AGP als MaxSAT

$$\text{Hard: } \bigvee_{i: V[i][j]=1} g_i \quad \forall j \qquad \text{Soft (Gewicht 1): } \neg g_i \quad \forall i$$

Jede verletzte Soft Clause ($g_i = 1$) kostet 1.

Solver minimiert Kosten = Anzahl aktiver Guards.

Keine Hilfsvariablen, kein iteratives k .

WCNF-Format: konkretes Beispiel (L-Polygon, $|P| = 8$)

c AGP MaxSAT, L-Polygon

h 1 2 3 6 7 8 0 % Cov. p_2

h 3 4 5 6 7 8 0 % Cov. p_4

h 1 3 4 5 6 7 8 0 % Cov. p_5

...

1 -1 0 % Soft: $\neg g_1$

1 -2 0 % Soft: $\neg g_2$

...

1 -8 0 % Soft: $\neg g_8$

Legende

h = Hard Clause (Coverage)

1 = Soft Clause, Gewicht 1

MaxHS minimiert:

$$\sum_i 1[g_i = 1]$$

= Anzahl aktiver Guards

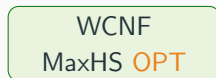
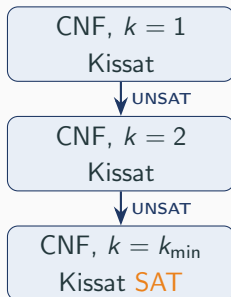
Lösung: MaxHS setzt $g_3 = 1$,

alle anderen = 0

$\Rightarrow$ Kosten = 1 = $k_{\min}$

(auch g_6, g_7, g_8 optimal)

Kissat (iterativ) vs. MaxHS (direkt)



Kissat: mehrere kleine CNF-Instanzen
Vorteil bei kleinem $k_{\min}$ (wenige UNSAT-Runden)

MaxHS: eine WCNF-Instanz
Vorteil bei großem $k_{\min}$ (viele teure UNSAT-Nachweise entfallen)

Polygon-Klassen

Polygon-Klassen: Was macht sie für SAT schwer?

comb_N: Kammstruktur



comb_3

- Jede Zacke braucht eigenen Guard
- $k_{\min} = N$ — wächst linear
- Kissat: $N-1$ UNSAT-Nachweise, Kosten explodieren
- MaxHS löst direkt in konstanter Zeit

random_N_seedM: Zufallspolygon



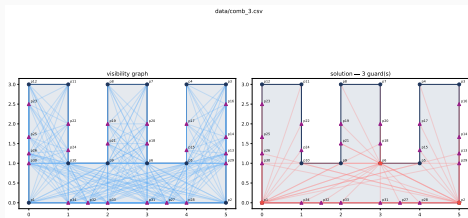
random_10

- Optimum oft klein ($k = 1$ oder $k = 2$)
- Kissat: wenige UNSAT-Runden nötig
- MaxHS trägt Overhead der Soft-Clause-Optimierung
- Kandidatenanzahl wächst stark mit N

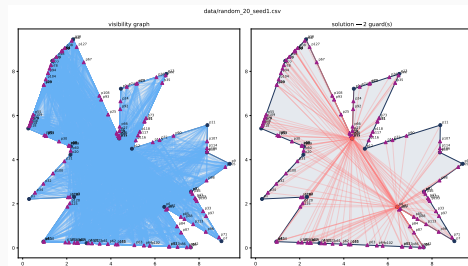
- **Instanzen:** comb_3 bis comb_6; random_10-30_seed1-5
- **Zweistufiges Benchmarking:**
 1. Preprocessing-Phase: Kandidaten + Sichtbarkeit berechnen, CNF/WCNF erzeugen und cachen
 2. Solve-Phase: reine Solver-Zeit auf gecachten Dateien messen
- **Kandidatengenerierung:** 0, 1, 2 Runden

Ergebnisse

Beispiellösungen: Sichtbarkeitsgraph & Guards



comb_3: optimale Lösung, $k_{\min} = 3$ Guards

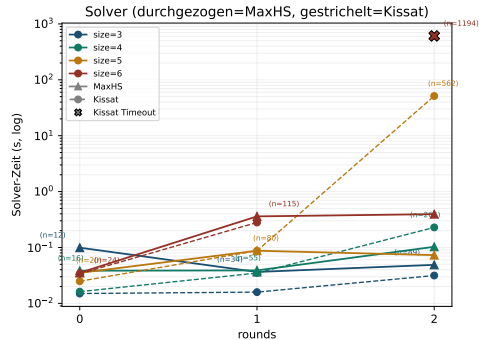
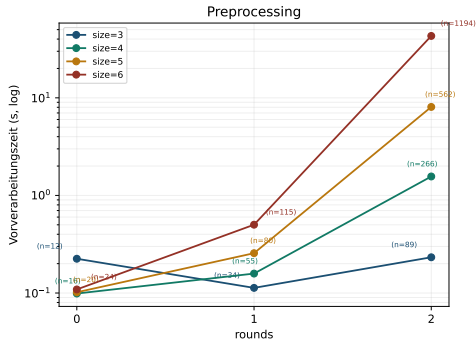


random_20_seed1: optimale Lösung, $k_{\min} = 2$
Guards

Sichtbarkeitsgraph aller Kandidaten mit gewählten Guards (rot) und ihrer Abdeckung.

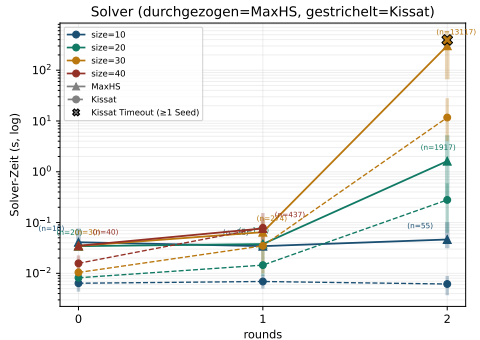
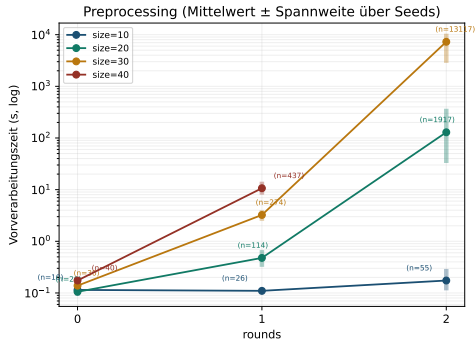
Einfluss der Rundenzahl: comb

comb: Zeit vs. rounds (eine Linie pro size, Kandidatenzahl in Klammern)

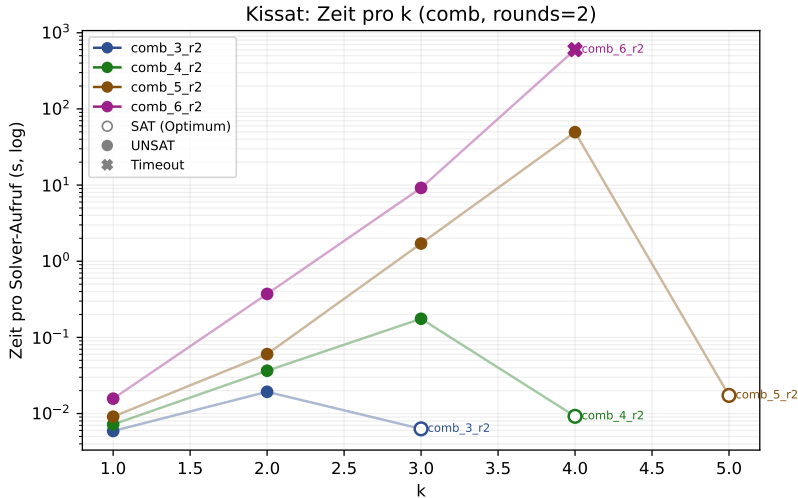


Einfluss der Rundenzahl: random

random: Zeit vs. rounds (eine Linie pro size, Kandidatenzahl in Klammern)



Ergebnisse: UNSAT-Explosion bei comb



Diskussion

Fazit & Ausblick

Zusammenfassung

- AGP erfolgreich als SAT (Sequential Counter) und MaxSAT (WCNF) enkodiert

Ausblick

- **Kandidatengenerierung** ist Flaschenhals bei großen Instanzen (random_30: Vorverarbeitung sprengt Zeitlimit) $\Rightarrow$ Optimierungspotenzial als nächster Schritt
- **Rundenvergleich**: 1 vs. 2 vs. mehr Runden — bessere Optima?

Vielen Dank!

Fragen?